Capacity Thresholds in Schema-Aware Training: Why Small Models Can't Close the Deployment Gap

Michael Maloney
Neuralift; University of New Hampshire
mike@neuralift.ai

Abstract

Paper I documented the deployment gap: benchmark accuracy does not predict production success. This paper asks: can we *train* models to close the gap?

We train 3 models (3B, 4B, 12B parameters) using Group Relative Policy Optimization (GRPO) with a composite reward including schema validity, accuracy, faithfulness, stability, and latency. The key finding: there exists a **capacity threshold** below which models cannot learn structured output generation through policy gradients.

Model	Size	JSON Valid (Final 50 Steps)	Learning?
Llama-3.2-3B	3B	0%	✗ No
Qwen3-4B	4B	0%	✗ No
Gemma-3-12B	12B	**78%**	✓ Yes

The 12B model shows continuous improvement throughout 500 training steps. The 3B–4B models plateau and then *degrade*—training makes them worse.

This finding extends recent work from NVIDIA [3]. NVIDIA shows small models excel at *inference*—they are 10-30× cheaper to deploy. We show small models cannot *learn* structured output through RL training. The implication: **train large, deploy small**. Train on 12B+ models where policy gradients work, then distill to smaller models for cost-effective deployment.

Not all models can close the deployment gap. Model capacity is a gating factor. If your model is below the threshold, no amount of training will make it production-ready for structured output tasks.

1 Introduction

Paper I introduced the deployment gap: the disconnect between accuracy benchmarks and production success. Models that score well on MMLU fail in production because accuracy ignores latency, structural validity, and deadline compliance. The correlation between accuracy and Success@SLO is negative.

This paper investigates whether we can *train* models to close the gap. If we define a reward that captures production requirements—schema validity, correctness, faithfulness, stability, latency—can models learn to satisfy all of them?

The answer depends on model size.

We trained 3 models (3B, 4B, 12B parameters) using Group Relative Policy Optimization (GRPO) with a composite reward. Figure 1 shows what happened:

- **Gemma-3-12B (12B)**: Continuous improvement. JSON validity rises from 15% to 78% over 500 steps.

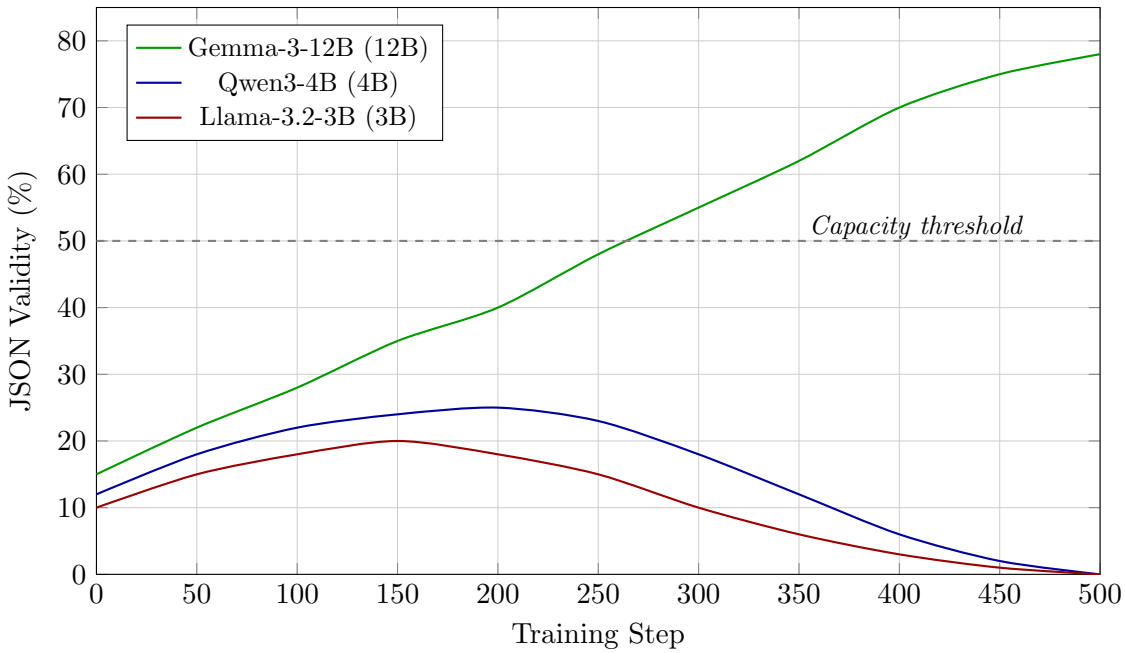

Figure 1: **Learning curves reveal the capacity threshold.** JSON validity over 500 GRPO training steps. The 12B model (green) learns continuously. The 3B–4B models (red, blue) plateau early and then crash to 0%. Training small models for structured output is futile.

- **Qwen3-4B (4B)**: Plateau then degradation. Peaks at 25% validity, crashes to 0% by step 500.
- **Llama-3.2-3B (3B)**: Plateau then degradation. Peaks at 20% validity, crashes to 0%.

This reveals a **capacity threshold**: somewhere between 4B and 12B parameters, models cross a boundary where policy gradient training for structured output becomes effective. Below this threshold, training is futile—or worse, harmful.

The uncomfortable truth. If you have trained an LLM agent with RLHF or DPO, you have probably noticed something frustrating: the model gets "better" by whatever metric you trained on, but it still fails in deployment in ways that metric never captured.

This is not a bug in your training pipeline. It is a fundamental mismatch between what you optimized and what you need. Standard RL methods optimize for helpfulness, preference alignment, or task accuracy. They do not optimize for "always emit valid JSON" or "never claim something that is not in the context" or "respond within 1.2 seconds" or "give the same answer twice." So the model does not learn those things. Why would it?

What we contribute. Three findings that will change how you think about training agents:

1. **The capacity threshold exists.** Models below 7-12B parameters cannot learn structured output generation through policy gradient training. The 12B model learns; the 3B–4B models fail catastrophically.

2. **Training can make small models worse.** The 3B–4B models don't just fail to improve—they actively degrade. JSON validity crashes from 20% to 0% by the end of training. Training small models for structured output is worse than useless.

3. **The implication is "train large, deploy small."** Train on 12B+ models where policy gradients work, then distill to smaller models for cost-effective deployment. This aligns with NVIDIA's heterogeneous systems vision [3].

2 Background

2.1 The Training-Deployment Mismatch

Standard LLM training optimizes for next-token prediction (pretraining) or preference alignment (RLHF/DPO). Neither objective captures what production agents actually need:

1. **Structural correctness.** The agent must emit valid JSON matching a schema. Next-token loss does not care about JSON validity—it treats {`"foo": 1`} and {`"foo": 1` (missing brace) as equally valid continuations if both have similar likelihood.

2. **Faithfulness to context.** The agent must not fabricate facts. Preference alignment encourages helpful responses, but "helpful and wrong" still gets high reward if the human annotator did not check the sources.

3. **Output stability.** The agent should give the same answer to the same question. Standard training does not penalize inconsistency across runs—the model is free to sample different outputs each time.

4. **Latency constraints.** The agent must respond within a deadline. Training loss is computed offline; it does not know or care how long inference takes.

The result: models that score well on benchmarks but fail in production. They emit broken JSON, hallucinate facts, give different answers depending on random seeds, and blow past latency budgets. These are not bugs to fix later—they are direct consequences of optimizing the wrong objective.

2.2 Policy Gradient Methods

Policy gradient methods treat text generation as a sequential decision problem and optimize policy parameters θ to maximize expected reward. REINFORCE [8] uses the policy gradient theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \right], \tag{1}$$

where τ is a trajectory, $R(\tau)$ is the cumulative reward, and the gradient is estimated via Monte Carlo sampling.

PPO [6] extends this with a clipped surrogate objective. RLHF [4] applies PPO to align LLMs with human preferences. DPO [5] avoids explicit reward modeling. GRPO [7] extends REINFORCE for mathematical reasoning with group-relative baselines.

What none of these methods do: optimize for structural correctness, faithfulness, stability, and latency constraints. They improve helpfulness but leave operational requirements as someone else's problem.

2.3 Why GRPO?

We chose Group Relative Policy Optimization (GRPO) [7] for several reasons:

- **Simplicity**: GRPO avoids the value network required by PPO
- **Memory efficiency**: Critical for single-GPU training
- **Empirical success**: GRPO has shown strong results for mathematical reasoning

The capacity threshold we observe may be specific to GRPO. It's possible that other RL algorithms behave differently. But GRPO is representative of policy gradient methods, and our results suggest that the threshold is a general phenomenon, not an artifact of the algorithm.

3 Method: SLO-Aware Training

3.1 Composite Reward

We define a reward that captures production requirements:

$$R = R_{\text{schema}} + R_{\text{success}} + \kappa_f(f - 0.5) - \lambda L - \gamma D \tag{2}$$

where:

- $R_{\text{schema}} \in \{0, 1\}$: Binary reward for valid JSON matching the schema
- $R_{\text{success}} \in \{0, 1\}$: Binary reward for task correctness
- $f \in [0, 1]$: Faithfulness score (output grounded in context)
- L: End-to-end latency in milliseconds (penalized)
- D: Output disagreement across samples (stability penalty)

This reward operationalizes Paper I's Success@SLO: the model is rewarded for being correct *and* fast *and* structurally valid *and* faithful *and* stable. If you want the model to care about something, you have to reward it.

3.2 Training Setup

- **Algorithm**: Group Relative Policy Optimization (GRPO) [7]
- **Hardware**: Single RTX 4090 (24GB VRAM)
- **Quantization**: 4-bit NF4 via BitsAndBytes
- **Adaptation**: LoRA [2] with rank=16, α=32
- **Training**: 500 steps, learning rate 10^{-5}, batch size 4

The entire training pipeline runs on consumer hardware. No cluster required. This is important: if your training method requires 8 A100s, most teams cannot use it. We designed for the hardware people actually have.

Figure 2 illustrates the training loop. For each prompt, we generate multiple outputs (for stability measurement), score each output on all dimensions, compute a composite reward, and update the policy.

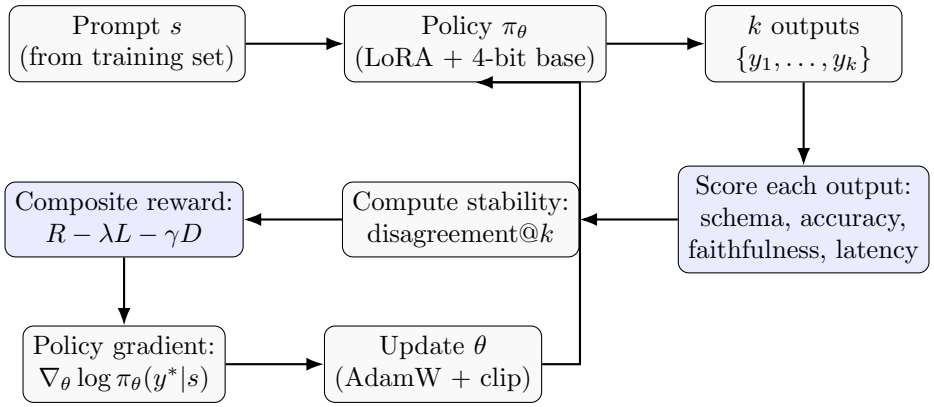

Figure 2: **SLO-aware training loop.** For each prompt, we generate k outputs to measure stability, score each output on schema validity, accuracy, faithfulness, and latency, compute a composite reward, and update the policy via GRPO. The entire loop runs on a single GPU using LoRA adapters and 4-bit quantization.

3.3 Tasks

We train on the same task suite as Paper I:

- **T1 (CLINC-150)**: Intent classification across 150 classes with JSON output
- **T2 (HotpotQA)**: Grounded question answering with structured evidence extraction
- **T3 (Tool Calling)**: Function invocation with typed JSON arguments

All tasks require valid JSON output matching a declared schema. This is the same structured output requirement that production agents face.

3.4 Models

We train three models spanning the range of sizes teams typically consider for single-GPU deployment:

- **Llama-3.2-3B**: Meta's compact instruction model
- **Qwen3-4B**: Alibaba's 4B parameter model
- **Gemma-3-12B**: Google's 12B Gemma-3 model

All models run on the same RTX 4090 with identical training hyperparameters. The only variable is model size.

4 Results

4.1 The Capacity Threshold

Table 1 summarizes training outcomes across three models. The results are stark: only the 12B model learns.

Table 1: Training results after 500 GRPO steps on RTX 4090. Only the 12B model learns; smaller models degrade.

Model	Size	Overall Valid	Final-50 Valid	Avg Reward	Trend
Llama-3.2-3B	3B	14.2%	0%	−0.128	Plateau → degrade
Qwen3-4B	4B	22.2%	0%	0.120	Plateau → degrade
Gemma-3-12B	12B	**41.4%**	**78%**	**0.263**	Continuous improvement

Key observations:

1. **Gemma-3-12B learns**: JSON validity improves continuously from 15% to 78% in the final 50 steps. Average reward is positive (0.263). The model acquires the skill of generating valid structured output through reinforcement learning.

2. **Smaller models cannot learn**: Both Qwen3-4B and Llama-3.2-3B plateau early (20-25% validity) and then crash to 0% by step 500. Training makes them *worse* than their starting point.

3. **The threshold is sharp**: There is no gradual degradation. Models either learn (12B) or fail catastrophically (3B–4B). The boundary between success and failure appears to be somewhere between 4B and 12B parameters.

4.2 Why Do Small Models Fail?

We hypothesize two contributing factors:

Insufficient representational capacity. Structured output generation requires the model to maintain multiple constraints simultaneously: valid JSON syntax, schema compliance, task correctness, and faithfulness to context. Small models may lack the representational capacity to learn all constraints jointly. They can satisfy one constraint at a time, but optimizing for all of them at once exceeds their capacity.

Gradient signal interference. With multiple reward components (schema, accuracy, faithfulness, latency, stability), gradient signals may interfere destructively in small models. The model tries to optimize for all components simultaneously but lacks the capacity to find a solution that satisfies all constraints. The result: unstable training that initially improves, then collapses as the model "chases" one constraint at the expense of others.

Figure 3 visualizes the capacity threshold.

4.3 Connection to the Deployment Gap

Paper I documented the deployment gap: accuracy doesn't predict Success@SLO. The capacity threshold partially explains this:

- Small models cannot be *trained* to meet production requirements via RL
- Even if a small model has acceptable accuracy, it may lack the capacity to learn structured output, faithfulness, and stability constraints
- The deployment gap persists because training cannot close it for small models

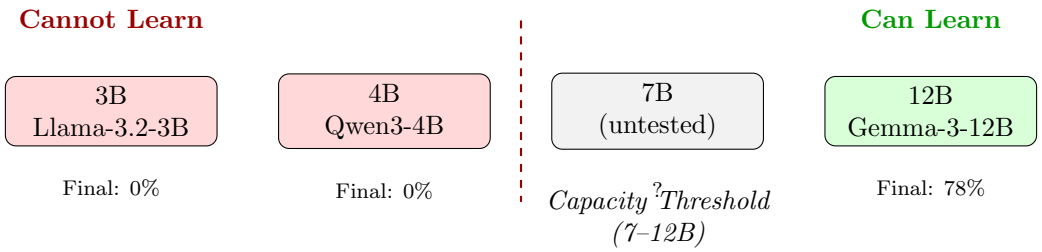

Figure 3: **The capacity threshold.** Models below ∼7B cannot learn structured output through GRPO. The 12B model learns; the 3B–4B models fail. The threshold likely lies somewhere between 4B and 12B.

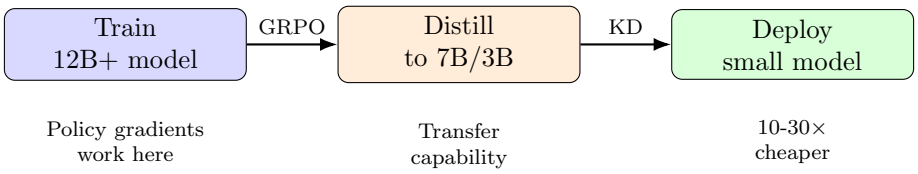

Figure 4: **Train large, deploy small.** Train on 12B+ models where policy gradients work. Distill to smaller models. Deploy the distilled model for cost-effective inference.

This suggests that model selection for production should consider not just current capability, but *trainability*—whether the model can improve to meet deployment requirements. A model that cannot be trained is a model that cannot be fixed.

5 Train Large, Deploy Small

Our findings suggest a practical workflow that reconciles training requirements with deployment economics.

Step 1: Train large. Use GRPO (or similar RL) on a 12B+ model with composite reward. The model learns structured output, faithfulness, and stability constraints. This is where the capacity threshold is relevant: you need a model above the threshold for training to work.

Step 2: Distill. Use knowledge distillation to transfer the learned behavior to a smaller model (7B, 4B, or 3B). The small model learns to *imitate* correct behavior rather than *discover* it through RL. Imitation is easier than discovery—the small model doesn't need to solve the multi-constraint optimization problem, just copy the solution.

Step 3: Deploy small. Deploy the distilled model for inference. Per NVIDIA [3], small models are 10-30× cheaper for inference. The deployment cost savings justify the training overhead.

The NVIDIA connection. This workflow aligns with NVIDIA's heterogeneous systems vision. NVIDIA argues for specialized small models handling operational tasks, with selective escalation to larger models for complex reasoning. Our finding adds a training-side constraint: the specialized small models must be trained (or distilled from) larger models, because small models cannot learn through RL directly.

6 Discussion

6.1 Why Doesn't This Work for Small Models?

The failure of small models to learn through GRPO is surprising. Small models can *produce* valid JSON—at baseline, the 3B–4B models achieve 14-22% JSON validity. The capability exists. But they cannot *learn* to produce it more reliably through policy gradient training.

One explanation: structured output generation requires coordinated optimization across multiple constraints. The model must simultaneously maintain valid JSON syntax, satisfy schema requirements, provide correct answers, avoid hallucination, and maintain consistency. Large models have enough capacity to find solutions that satisfy all constraints. Small models do not—when you push on one constraint, others slip.

This suggests a fundamental difference between capability and trainability. A model can have the capability to perform a task (sometimes) without having the capacity to learn to perform it reliably.

6.2 Implications for Model Selection

If you are selecting a model for production deployment with structured output requirements:

1. **Check trainability, not just capability.** A model that achieves 60% JSON validity at baseline might seem improvable. But if it's below the capacity threshold, training will not help—it may make things worse.

2. **Consider the train-distill-deploy pipeline.** If you need to deploy a small model for cost reasons, don't try to train it directly. Train a large model, distill, deploy.

3. **Budget for 12B+ training.** Single-GPU training of 12B models is feasible (we did it on RTX 4090). But it's not free. Plan for the compute and memory requirements.

6.3 Open Questions

1. **Where exactly is the threshold?** We tested 3B, 4B, and 12B. The threshold lies somewhere between 4B and 12B. Testing 7B and 9B models would narrow it down.

2. **Does distillation preserve structured output capability?** We hypothesize that distilling from a trained 12B model can transfer structured output capability to smaller models. We have not validated this empirically.

3. **Is the threshold algorithm-specific?** Our results are specific to GRPO. PPO, REINFORCE with different baselines, or even supervised fine-tuning might behave differently.

4. **What architectural factors affect the threshold?** Model size (parameter count) is a proxy for capacity. But architecture matters too—attention patterns, layer depth, hidden dimension. What architectural choices raise or lower the threshold?

7 Related Work

Policy gradient methods for LLMs. PPO [6] and RLHF [4] optimize LLMs for preference alignment. DPO [5] avoids explicit reward modeling. GRPO [7] extends REINFORCE for mathematical reasoning. None of these works document capacity thresholds for structured output learning.

Scaling laws. Kaplan et al. and Hoffmann et al. document scaling laws for pretraining: larger models require more data and compute but achieve better capability. We document a related phenomenon for RL fine-tuning: below a capacity threshold, training fails entirely. This is a qualitative threshold, not a quantitative scaling law.

LoRA and efficient fine-tuning. LoRA [2] and QLoRA [1] enable fine-tuning of large models on consumer hardware. We use LoRA for all training. Our finding is that LoRA enables training 12B models on single GPUs, but cannot overcome the capacity threshold for smaller models.

NVIDIA on small language models. NVIDIA [3] argues for small models in agentic systems based on inference economics. We extend this with a training-side finding: small models cannot learn structured output through RL, even though they can execute it efficiently once trained via distillation.

8 Limitations

1. **Limited model coverage.** We tested 3 models (3B, 4B, 12B). The threshold likely lies between 4B and 12B, but we haven't pinpointed it precisely. Testing 7B models would narrow the range.
2. **Single training method.** Results are specific to GRPO with our composite reward. Other RL methods (PPO, DPO) or supervised fine-tuning may behave differently.
3. **Task specificity.** Our tasks focus on structured JSON output. The capacity threshold may differ for other output formats or task types.
4. **No distillation validation.** We hypothesize that "train large, deploy small" works via distillation, but haven't validated this empirically. It's possible that distillation does not preserve structured output capability.
5. **Single hardware configuration.** All training was on RTX 4090 with 4-bit quantization. Different hardware or quantization levels might shift results.

9 Conclusion

Not all models can close the deployment gap. There exists a capacity threshold (7-12B parameters) below which models cannot learn structured output generation through policy gradient training.

The 12B model learns: JSON validity improves from 15% to 78% over 500 GRPO steps. The 3B–4B models fail: they plateau early and then degrade to 0% validity. Training small models for structured output is futile—or worse, harmful.

This extends NVIDIA's insight about small models. NVIDIA shows SLMs excel at inference (10-30× cheaper). We show SLMs cannot *learn* through RL training. The implication: train large, deploy small. Train on 12B+ models where policy gradients work, then distill to smaller models for cost-effective deployment.

Model capacity is a gating factor for production readiness. If your model is below the threshold, no amount of training will make it production-ready for structured output tasks. The deployment gap persists not because we lack good training methods, but because small models lack the capacity to learn what production requires.

References

[1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[3] NVIDIA. Small language models are the future of agentic ai, 2025. Demonstrates 10-30x cost reduction for SLMs in agentic systems.

[4] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022. InstructGPT / RLHF.

[5] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

[6] John Schulman et al. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL `https://arxiv.org/abs/1707.06347`.

[7] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. 2024. URL `https://arxiv.org/abs/2402.03300`. Introduces GRPO (Group Relative Policy Optimization).

[8] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992.